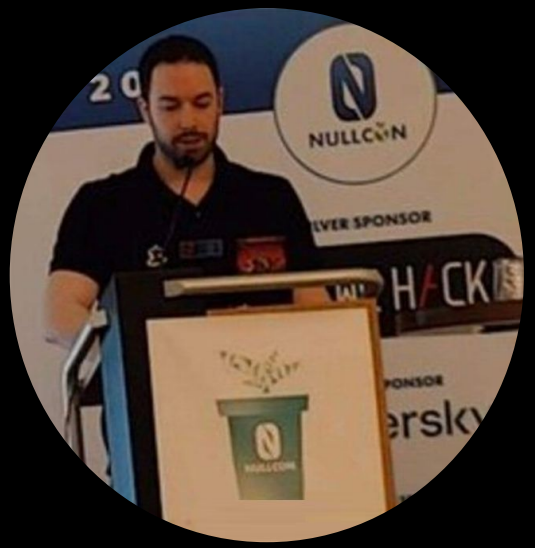
Breaking Secure Boot

Exploiting GRUB2's
(and other bootloaders)

Jonathan Bar Or (“JBO”)
SECCON 14

自己紹介



- ジョナサン・バー・オー (Jonathan Bar Or, “JBO”)
- 元マイクロソフト、現在クラウドストライク所属
- これまでに多数の脆弱性を報告
  - Windows, Linux, macOS,
  - Android, iOS, OT, ブートローダ
  - ほか多数
- https://jonathanbaror.com

目次

- Pre-UEFI ブートローダの問題点
- UEFI アーキテクチャ概要
- GRUB2 および shim の構成
- 発見した脆弱性
- デモ
- その他のブートローダ
- まとめ

昔の BIOS

- BIOS はブート ROM からロードされ、POST を実行
  - リアルモード（16-bit、セグメンテーションあり）
- BIOS は IVT に有用な割り込みハンドラを登録
- ブート順に基づき MBR を探し、ロードします
  - ブートローダ第1段は 0000:7C00 にロードされます
  - 通常は INT 13h を使用してディスクから第2段を読み込む
  - 最終的に、ブートローダがファイルシステム上のカーネルをロードします
- では、どこに問題があるでしょうか

BIOSの問題

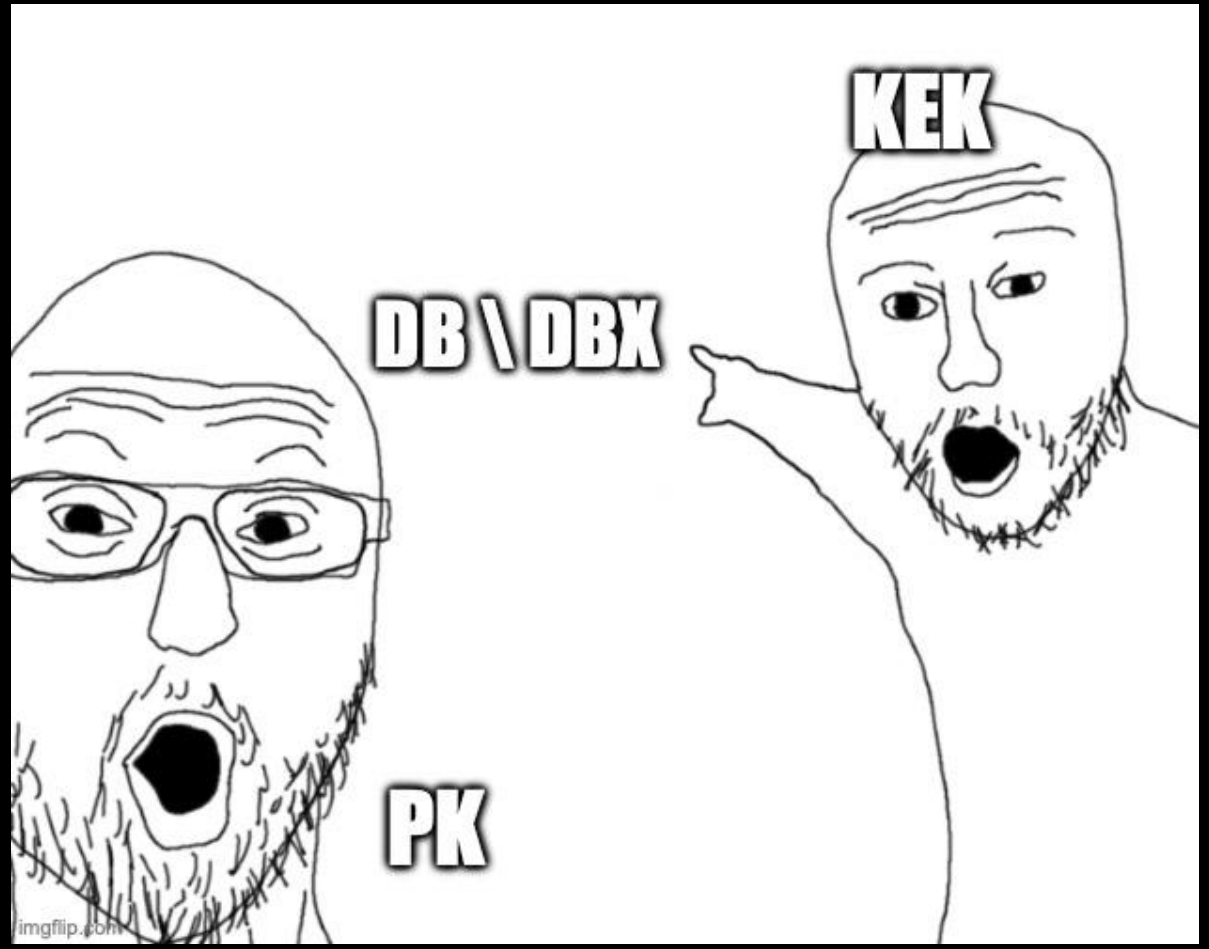
- 標準化されていません
- BIOS のアップデートは専用ソフトウェアを使用して行われます
- ブートキット
  - ファームウェアによるMBRの検証は行われません
  - 多くの場合、ファームウェア自体の検証は行われていません
  - 一般的には、IVTの変更を通じて INT 13h をフックします
- BIOS コード自体にバグがあったり、
  脆弱性が存在する可能性があります

ブートキットがどのように機能していたか

- MBRを感染させます
- 複数の割り込みをフックしてからOSをロードする
  - INT 13h（disk）：OSにさらなる悪意のあるコードを読み込ませます
  - INT 15h（system）：悪意のあるコードを挿入したり、BIOSメモリを改変したりします
  - INT 1Ah（Clock/CMOS）：後でコードを実行します

UEFI入門

- Unified Extensible Firmware Interface
- サービスの標準化とモジュール化
- 信頼の根源
  - Platform Key → Key Exchange Key → DB と DBX エントリー
  - 信頼できるモジュールと信頼できないモジュール、およびブートローダーが含まれています
  - セキュアブートは、すべてが正常であることを確認します


KEK
DB \ DBX
PK
imgflip.com

UEFI DBX
UEFI DB
imgflip.com

UEFIにもセキュリティ上の問題がある

- 例えば、ESET 社による2025年1月のレポート
- CVE-2024-7344:

```
EntryPoint = 0LL;
SourceBuffer = sub_10008660();
CopyMem(DestinationBuffer, SourceBuffer, 0x60uLL);
// manually load PE image
Status = CustomLoadImage(SourceBuffer, &fdata, &fsize, &EntryPoint);
if ( Status >= 0 )
{
  ImageHandle = sub_10008650();
  // Execute entry point of a loaded image
  Status = EntryPoint(ImageHandle, gST);
  CopyMem(SourceBuffer, DestinationBuffer, 0x60uLL);
  return Status >= 0;
}
else
{
  sub_10009840("UEFI\\RxPE.cpp", 266LL, "Handle image failed!state:0x%lx", Status);
  return 0;
}
```

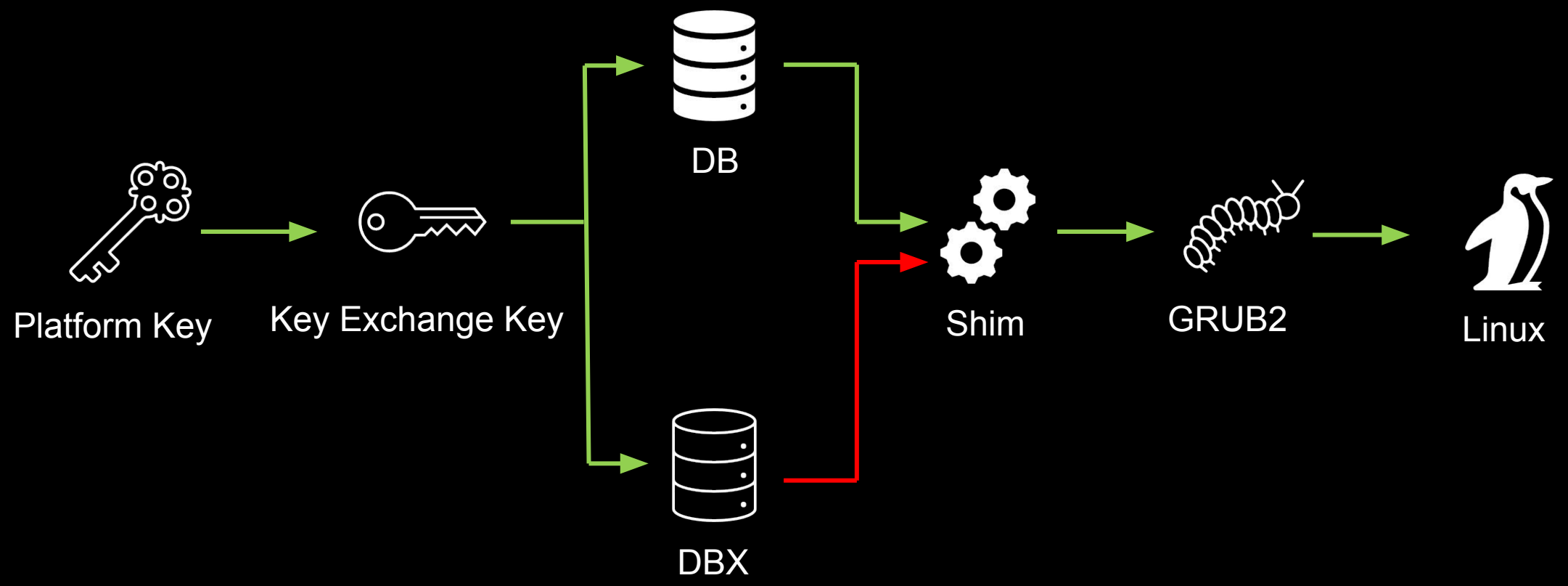
```
InitFstruc(FStruc);
FOPEN_SUCCESS = 0;
qmemcpy(filenames, L"\\EFI\\Microsoft\\boot\\cloak64.dat", 0x40uLL);
memset(&filenames[8], 0, 0x1C8uLL);
qmemcpy(v4, L"\\EFI\\boot\\cloak64.dat", 0x2CuLL);
memset(&v4[44], 0, 0x1DCuLL);
qmemcpy(v5, L"\\EFI\\Microsoft\\boot\\cloak.dat", 0x3CuLL);
memset(&v5[60], 0, 0x1CCuLL);
qmemcpy(v6, L"\\EFI\\boot\\cloak.dat", 0x28uLL);
memset(&v6[40], 0, 0x1E0uLL);
for ( i = 0; i < 4; ++i )
{
  // Try opening cloak file (break on success)
  if ( FOpen(FStruc, &filenames[65 * i], 0x2000u) )
  {
    FOPEN_SUCCESS = 1;
    break;
  }
}
if ( FOPEN_SUCCESS )
{
  *dataSize = FGetSize(FStruc);
  if ( *dataSize > 0x200uLL )
  {
    FSetStartOfFile(FStruc);
    if ( FReadCloakHdr(FStruc, &CloakHdr, 0x200u) == 0x200LL )
    {
      // Check if it contains ALRM signature
      if ( CloakHdr.signature == 'MRLA' )
      {
        // process the header, read and decrypt an embedded PE
        if...
      }
      else
      {
        ErrPrint("%u: Invalid file!\n", 78LL);
```

GRUB2

- GRUB2 はブートローダー（主に Linux で使用される）です
  - 完全にオープンソースです
  - 多くの興味深いファイルシステムをサポートしています
  - 複雑な機能（フォント、ネットワーク接続、画像）を備えています
  - 複数のアーキテクチャ（MIPS、ARM、x86）に対応しています
- オープンソースであるため、セキュアブートに関しては課題が生じます
  - OEMメーカーは、GRUBを検証した後にロードし、承認済みの小型の「シム（shim）」を用意しています

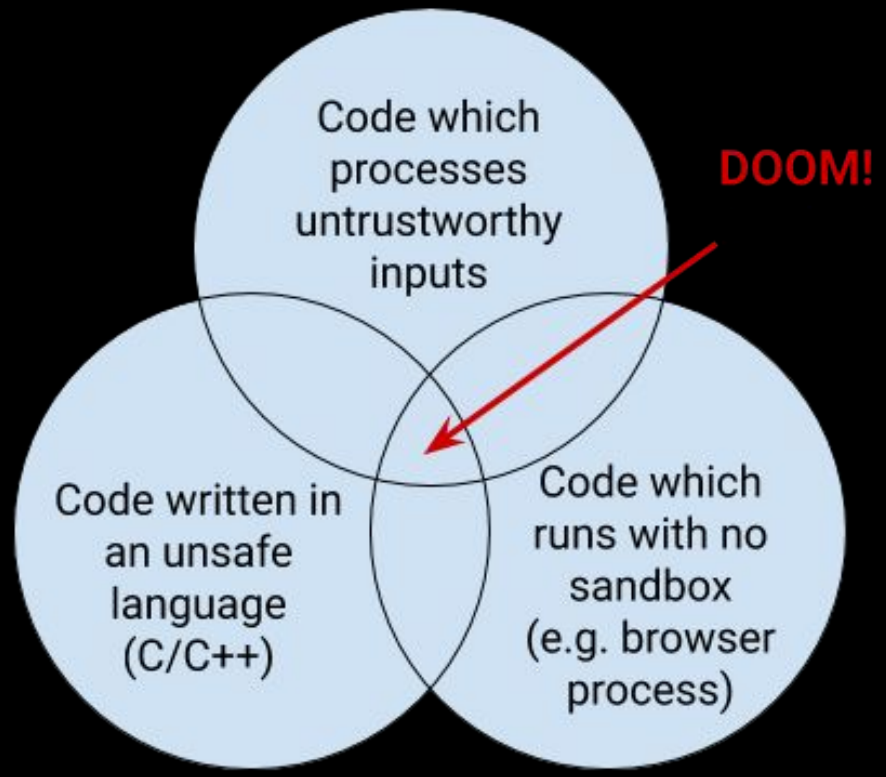
DOOM
Arch Linux
Advanced Options for
UEFI Settings
id SOFTWARE
id SOFTWARE, ©1993

ここまでのまとめ



GRUB2 の何が問題ですか

- Google の「Rule of two」:



GRUB2の何が問題ですか

- OSがないのでサンドボックス機能がありません！
  - ASLR、DEP、その他いかなる種類のメモリ保護もありません
  - スタッククッキーは使用されていません（ほとんどのビルドにおいて）
- 信頼できない大規模な入力を処理します
  - ファイルシステム
  - グラフィック（画像とフォント）
  - ネットワーク
- C言語でプログラムされています
  - 優れたRust製ブートローダーはいくつか存在するものの、DBに特定のキーを手動で追加しない限り、どれもセキュアブートによって信頼されることはありません

過去の事例

CVE(s)	Subsystem(s)	Root cause
CVE-2020-10713	Configuration file	Buffer overflow in the parsing of the GRUB2 configuration file (grub.cfg).
CVE-2021-3695 CVE-2021-3696 CVE-2021-3697	Image parsing	Several buffer overflow vulnerabilities in media image parsing.
CVE-2022-28733 CVE-2022-28734	Network	Various buffer overflow vulnerabilities when parsing IP or HTTP packets.
CVE-2022-28735	Shim	Non-kernel files could be loaded and execute arbitrary code.
CVE-2023-4692	NTFS (filesystem)	Heap out-of-bounds was discovered in the NTFS implementation for GRUB2.

grub-emu を使ったファジング

- GRUB2 のビデオ機能とグラフィック機能のファジングを開始しました。
- ブートローダーのファジングは難しい！
  - 幸いなことに、「grub-emu」があります！
  - AFL++で動作させるために、GRUB のソースコードに軽微な修正を加える必要がありました。
  - 非常に遅いですが、動作します

```
    american fuzzy lop ++4.00c {default} (./grub-core/grub-emu) [fast]
┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 31 sec       │  cycles done : 0      │
│   last new find : none seen yet                      │ corpus count : 4      │
│last saved crash : none seen yet                      │saved crashes : 0      │
│ last saved hang : none seen yet                      │  saved hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 0.0 (0.0%)        │    map density : 0.04% / 0.04%         │
│  runs timed out : 0 (0.00%)         │ count coverage : 1.01 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : trim 512/512          │ favored items : 1 (25.00%)             │
│ stage execs : 46/197 (23.35%)       │  new edges on : 1 (25.00%)             │
│ total execs : 297                   │ total crashes : 0 (0 saved)            │
│  exec speed : 7.97/sec (zzzz...)    │  total tmouts : 0 (0 saved)            │
├─ fuzzing strategy yields ───────────┴───────────────┬─ item geometry ───────┤
│   bit flips : disabled (default, enable with -D)    │    levels : 1         │
│  byte flips : disabled (default, enable with -D)    │   pending : 2         │
│ arithmetics : disabled (default, enable with -D)    │  pend fav : 1         │
│  known ints : disabled (default, enable with -D)    │ own finds : 0         │
│  dictionary : n/a                                   │  imported : 0         │
│havoc/splice : 0/0, 0/0                              │ stability : 99.80%    │
│py/custom/rq : unused, unused, unused, unused        ├───────────────────────┘
│    trim/eff : n/a, disabled                         │          [cpu000: 41%]
└─────────────────────────────────────────────────────┘
```

ガイド付きコード監査

- 私はファイルシステムの実装を調べ始めました
- LLMやその他のツール（CodeQL など）を使って少し作業をしています
- 多くの危険な操作
  - 中には偽陽性もあります

```c
static grub_err_t
grub_jfs_lookup_symlink (struct grub_jfs_data *data, grub_uint32_t ino)
{
  grub_size_t size = grub_le_to_cpu64 (data->currinode.size);
  char *symlink;

  if (++data->linknest > GRUB_JFS_MAX_SYMLNK_CNT)
    return grub_error (GRUB_ERR_SYMLINK_LOOP, N_("too deep nesting of symlinks"));

  symlink = grub_malloc (size + 1);
  if (!symlink)
    return grub_errno;
  if (size <= sizeof (data->currinode.symlink.path))
    grub_memcpy (symlink, (char *) (data->currinode.symlink.path), size);
  else if (grub_jfs_read_file (data, 0, 0, 0, size, symlink) < 0)
    {
      grub_free (symlink);
      return grub_errno;
    }

  symlink[size] = '\0';

  /* The symlink is an absolute path, go back to the root inode.  */
  if (symlink[0] == '/')
    ino = 2;

  grub_jfs_find_file (data, symlink, ino);

  grub_free (symlink);

  return grub_errno;
}
```

発見した脆弱性

Component	Issue
UFS filesystem	Buffer overflow in symlink handling due to an integer overflow in allocation.
Squash4 filesystem	Buffer overflow in file reads due to an integer overflow in allocation.
ReiserFS filesystem	Buffer overflow in symlink handling due to an integer overflow in allocation.
JFS filesystem	Buffer overflow in symink handling due to an integer overflow in allocation.
RomFS filesystem	Buffer overflow in symlink handling due to an integer overflow in allocation.
UDF filesystem	Buffer overflow in block reads of UDF due to an out-of-bounds operation.
HFS filesystem	Buffer overflow in mounting due to *strcpy* on a non-NUL-terminated string.
HFS compressed	Buffer overflow in file opens due to an integer overflow in allocation.
Cryptography	Timing side-channel attack due to non-constant time memory comparison.
Dump command	Arbitrary memory reads (should be disabled in production).
Read command	Buffer overflow in line reads due to an integer overflow in allocation.

GRUB2のヒープ

- 非常にシンプルで、dlmalloc を彷彿させます
  - GRUB2 はシングルスレッドなので、それは当然のことです
  - 内部的に UEFI から領域ごとに新しいメモリを割り当てます
- シンプルなメタデータ構造を使用します
  - 次の空きブロックへのポインター
  - 現在のブロックサイズ
  - Magic (either free or occupied)
  - パディング（メモリ配置の調整のため）

複雑なヒープ・グルーミングを行う必要はない

- 割り当ては非常に予測可能な場所に存在します
  - 割り当てられたチャンクのオフセット 0x2130 の位置に、関数ポインターを見つけました
  - すべてのファイルシステム実装は、操作のための関数ポインターを備えています
    - Mount (“open”)
    - List files (“dir”)
    - Read
    - Label
    - Close
    - Search (時々サポートされる)

```
static struct grub_fs grub_ntfs_fs =
  {
    .name = "ntfs",
    .fs_dir = grub_ntfs_dir,
    .fs_open = grub_ntfs_open,
    .fs_read = grub_ntfs_read,
    .fs_close = grub_ntfs_close,
    .fs_label = grub_ntfs_label,
    .fs_uuid = grub_ntfs_uuid,
#ifdef GRUB_UTIL
    .reserved_first_sector = 1,
    .blocklist_install = 1,
#endif
    .next = 0
};
```

エクスプロイト

- ターゲットファイルシステムで整数オーバーフローを引き起こします
- 「close」機能の関数ポインタをオーバーライドします
- NOP スレッドを挿入します（すべてのメモリが RWX です）
- UEFI サービスのポインタさえも見つけることができました！
- 独自の「ブートローダー」を作成します

```
root@jbo-vm-nix:~/powergrub#
```

コードの類似性を見つける

- ブートローダーのコードリポジトリには、コードの重複が多数見られます
- LLM を使用して、モジュール間の類似点を見つける
  - “Which bootloaders exhibit similar implementations to GRUB’s filesystem modules?” って質問しました

コードの類似性を見つける

1. U-Boot

- **Overview:** Common on embedded devices, ARM, MIPS, PowerPC.
- **Filesystem modules:** U-Boot has **loadable filesystem drivers**, though they are usually compiled in rather than dynamically loaded at runtime like GRUB.
- **Similarity:** Conceptually similar in that you have separate code to handle `FAT`, `ext`, `UBIFS`, `JFFS2`. The modularity is at the source level rather than `.mod` runtime modules.
- **Reference:** Check `drivers/filesystems/` in U-Boot source.

---

2. Barebox

- **Overview:** Seen as the “modern GRUB for embedded Linux.”
- **Filesystem modules:** Barebox uses **modules** for filesystems (FAT, ext, UBIFS, etc.) which can be loaded from storage.
- **Similarity:** Very close to GRUB2—modular, loadable filesystem drivers, often dynamically loaded.
- **Reference:** `fs/` directory in Barebox source.

U-boot

CVE	Issue
CVE-2025-26726	Buffer overflow during directory table parsing in the SquashFS filesystem
CVE-2025-26727	Buffer overflow during inode parsing in the SquashFS filesystem
CVE-2025-26728	Buffer overflow during file reading in the SquashFS filesystem
CVE-2025-26729	Buffer overflow during symlink handling due to an integer overflow in the EroFS filesystem

Barebox

CVE	Issue
CVE-2025-26721	Buffer overflow during file creation in the persistent storage
CVE-2025-26722	Buffer overflow during symlink handling due to an integer overflow in the SquashFS filesystem
CVE-2025-26723	Buffer overflow during symlink handling due to an integer overflow in the EXT4 filesystem
CVE-2025-26724	Buffer overflow during symlink handling due to an integer overflow in the CramFS filesystem
CVE-2025-26725	Buffer overflow during directory entry parsing in the JFFS2 filesystem

まとめ

- GRUB2 のメンテナーにこれらの脆弱性を開示しました
  - 彼らと協力してバグを修正しました
  - 私自身もコードを提供しました
  - OEM各社は DB と DBX をアップデートしました
- Barebox とU-bootはそれぞれ独自に修正プログラムを展開しました
- オープンソースソフトウェアは、より厳密な監視が必要かもしれません

本日はありがとうございました